ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

CARRERA DE INGENIERÍA DE SOFTWARE

APLICACIONES MÓVILES

TALLER 1: REPORTE

Integrantes:

Dayanna Bravo

Esteban Gómez

Andreina Pallo

DOCENTE:

Vicente Eguez Sarzosa

CURSO:

GR1SW

FECHA DE ENTREGA: 15-04-2026

Quito - Pichincha - Ecuador

2026 A

Parte 1: El Viaje Arquitectónico del Desarrollo Móvil

1. Desarrollo Nativo Puro

Android (Google)

- **2008 (El Origen):** Lanzamiento de Android 1.0. Se utilizaba Java como lenguaje principal y Eclipse (con el plugin ADT) como entorno de desarrollo. La máquina virtual era Dalvik, diseñada para dispositivos con muy poca RAM.
- **2013 - 2014 (La Transición):** Google presenta Android Studio (basado en IntelliJ) para reemplazar a Eclipse. Se introduce ART (Android Runtime) en Android 4.4 (KitKat) para mejorar el rendimiento sustituyendo a Dalvik.
- **2017 (La Revolución):** En el Google I/O, Kotlin es declarado lenguaje oficial de primer nivel.
- **2019 (Kotlin First):** Google anuncia que el desarrollo de Android será prioritariamente en Kotlin. Lanzamiento de Jetpack Compose (en alpha), cambiando el paradigma de XML a UI Declarativa.
- **2024 - 2026 (Actualidad):** Kotlin 2.0 con el compilador K2 es el estándar. Java se mantiene por retrocompatibilidad en la versión 21+, pero casi todo el desarrollo nuevo es Kotlin.

iOS (Apple)

- **2007 (iPhone OS 1 - El Kernel Darwin):** Lanzamiento del iPhone original. No existían apps de terceros. Basado en una versión reducida de macOS (Kernel BSD/Darwin). La interfaz se gestionaba con la librería interna LayerKit (luego llamada Core Animation).
- **2008 (iPhone OS 2.0 - El Big Bang):** Apple lanza el SDK oficial. Los desarrolladores obtienen acceso a Objective-C y a las herramientas de Xcode. Nace la App Store.
- **2010 (iOS 4 - El cambio de nombre):** El sistema deja de llamarse "iPhone OS" para reflejar su llegada al iPad. Se introduce la Multitarea, lo que obliga a los

desarrolladores a gestionar el ciclo de vida de la app (Background vs Foreground) para no agotar la batería.

- **2011 (iOS 5 - Gestión de Memoria):** Introducción de ARC (Automatic Reference Counting).
    - Impacto: Antes, los desarrolladores usaban retain y release. ARC permitió que el compilador insertara esas instrucciones automáticamente, reduciendo drásticamente los errores de fuga de memoria (memory leaks).
- **2014 (Swift 1.0 - El Nuevo Paradigma):** Apple lanza Swift en la WWDC. Un lenguaje moderno, tipado y seguro que busca jubilar a Objective-C.
- **2019 (SwiftUI - UI Declarativa):** Se lanza SwiftUI, permitiendo construir interfaces con mucho menos código y de forma reactiva (el estado de la app dicta lo que se ve en pantalla).
- **2024 - 2026 (Swift 6 y VisionOS):** Swift 6 se consolida con un enfoque masivo en el manejo de la concurrencia segura (Data Race Safety). La arquitectura nativa se expande a VisionOS para computación espacial.

2. **La Era del WebView / Híbridos (Cordova, PhoneGap, Ionic inicial)**

En esta arquitectura, la aplicación no es más que un "contenedor" nativo vacío que abre un navegador web interno (WebView) a pantalla completa. El código de la lógica y la interfaz no es Java ni Swift, sino HTML5, CSS3 y JavaScript.

- **2008 (El Experimento):** Nitobi crea PhoneGap en un hackathon. Su objetivo era demostrar que se podía acceder a la cámara de un iPhone usando solo JavaScript.
- **2011 (La Institucionalización):** Adobe compra Nitobi (y con ello PhoneGap). Para mantener el espíritu abierto, donan el motor principal a la Apache Software Foundation, naciendo Apache Cordova.
- **2013 (La Revolución de la UI):** Max Lynch y Ben Sperry lanzan Ionic Framework (v1). [Actualmente, está muerta esta versión]

- Hito: Antes de Ionic, las apps híbridas se veían como webs feas. Ionic trajo una librería de componentes CSS que imitaban perfectamente los botones y menús de iOS y Android. Funcionaba sobre AngularJS.

- **2015 (La Crisis de Rendimiento):** Mark Zuckerberg declara famosamente: "El mayor error que cometimos como empresa fue apostar demasiado por HTML5 en lugar del nativo". Esto marcó el inicio del fin de la hegemonía de los híbridos puros.

- **2020 (El Cierre):** Adobe anuncia oficialmente que deja de dar soporte a PhoneGap y a sus servicios de compilación en la nube (PhoneGap Build).

- **2024 - 2026 (La Evolución):** Cordova entra en modo mantenimiento. Ionic sobrevive desplazando a Cordova por su propio motor llamado Capacitor.

3. **El Puente Nativo-JavaScript y otros puentes**

A diferencia de Cordova (que renderiza HTML), esta arquitectura utiliza JavaScript para la lógica, pero le pide al sistema operativo que dibuje botones, listas y mapas reales de iOS y Android.

- **2008 - 2009 (Titanium Appcelerator - El Pionero):** Lanzado casi al mismo tiempo que PhoneGap, pero con una idea distinta: en lugar de un WebView, usaba un Proxy Nativo. Fue la primera vez que se pudo crear una app con JS que usara un UIButton real de iOS.

- **2011 (Appcelerator Titanium):** El precursor olvidado. Fue el primero en intentar usar JS para controlar componentes nativos, pero su implementación era inestable y su ecosistema cerrado. Auge de la plataforma, posicionándose como una alternativa fuerte para desarrollo multiplataforma. [Actualmente obsoleto]

- **2014 (NativeScript - El Acceso Total):** Presentado por Telerik. Su innovación fue eliminar la necesidad de "plugins" manuales. Permitía llamar a cualquier API de Android o iOS directamente desde JavaScript usando un motor de reflexión. [Estado actual: nicho/especializado]

- **2015 (React Native - El Gran Cambio):** Facebook lanza React Native en versión 0.1. Cambió la industria al introducir el paradigma "Learn once, write anywhere".

- **2018 - 2020 (La Crisis del Bridge):** Airbnb y Udacity publican artículos famosos explicando por qué abandonaban React Native. El problema: el "Puente" (Bridge) era un cuello de botella para animaciones fluidas y gestos complejos.
- **2021 - 2024 (La Nueva Arquitectura):** Meta reescribe el núcleo de React Native. Introducen JSI (JavaScript Interface) y Fabric para eliminar el puente JSON asíncrono.
- **2026 (Estado Actual):** React Native (v0.7x - 0.80+) es el estándar para apps corporativas. Se ha movido casi totalmente hacia una comunicación síncrona basada en C++.

4. El Motor de Renderizado Propio

En esta arquitectura, el framework no le pide al sistema operativo un "Botón". En su lugar, pide un lienzo en blanco (Canvas) y utiliza su propio motor gráfico para dibujar cada píxel del botón, la sombra y la animación. Es, en esencia, cómo funciona un motor de videojuegos (como Unity), pero optimizado para interfaces de usuario.

- **2014 (Proyecto Sky):** Un grupo de ingenieros de Google comienza a experimentar con el motor de renderizado de Chrome (Blink) para ver qué tan rápido podía ser. Logran alcanzar los 120 FPS (cuadros por segundo).
- **2015 (Anuncio oficial):** Se presenta en el Dart Developer Summit como "Sky", prometiendo una interfaz que nunca se detiene ("never-jank").
- **2017 (Flutter Alpha):** El proyecto cambia de nombre a Flutter. Se elige Dart como lenguaje por su capacidad de compilación rápida y su manejo eficiente de objetos de corta vida (como los widgets).
- **2018 (Flutter 1.0):** Lanzamiento de la primera versión estable. La industria se sorprende: una app de Flutter se ve exactamente igual en un Android de gama baja que en un iPhone Pro.
- **2021 (Flutter 2.0):** Expansión total. Flutter deja de ser solo para móviles y se vuelve "portable", soportando Web, Desktop y sistemas embebidos con el mismo código.
- **2023 - 2024 (La Era de Impeller):** Google lanza Impeller, un nuevo motor de renderizado diseñado desde cero para sustituir a Skia.

- Hito: Se soluciona el "Shader Compilation Jank" (pequeños tirones en las animaciones de iOS al abrir la app por primera vez).

- **2026 (Estado Actual):** Flutter (v3.25+) es el líder indiscutible en desarrollo multiplataforma para startups y apps con diseños altamente personalizados. Soporta WebAssembly (Wasm) de forma nativa, llevando el rendimiento web a niveles casi nativos.

5. **El Futuro: Foldables, AR y Wearables**

Esta fase rompe con 15 años de diseño basado en pantallas estáticas. Ya no se diseña para un "teléfono", sino para un ecosistema de superficies dinámicas.

Foldables

- **Tecnologías Clave:** * Jetpack WindowManager (Android): Permite detectar la "postura" del dispositivo (modo mesa, modo libro).
    - SwiftUI Adaptive Layouts: Uso de ViewThatFits para cambiar la UI dinámicamente.
- Estado: **En auge**. Es crítico para apps de productividad (Office, Editores de video).

AR/XR

En Realidad Aumentada, el software debe procesar la imagen de la cámara y superponer un objeto 3D en menos de 20ms.

- **El Problema Técnico:** Los frameworks como React Native o Flutter tienen un "costo de paso" (bridge) que añade latencia.
- **La Solución:** * Motores Dedicados: Uso de Unity (C#) o RealityKit (Swift).
    - Cálculo Espacial: Procesamiento de matrices en la GPU para evitar que el objeto "tiemble" en la pantalla.
- Veredicto de Relevancia: El desarrollo Nativo Puro recupera aquí su liderazgo absoluto debido a la necesidad de acceso directo al sensor LiDAR y la GPU.

Wearables

Su futuro es la independencia.

- **Tecnologías**: * WatchOS (Swift): Uso de Complications y Push Notifications enriquecidas.
    - Wear OS (Kotlin): Basado en tiles (mosaicos) para acceso rápido.

- **Detalle Técnico**: Se utilizan protocolos de comunicación de bajo consumo como Bluetooth LE (Low Energy) y arquitecturas de sincronización de datos asíncronas.

Parte 2: Análisis Crítico de Aplicaciones

- Instagram

Instagram es una aplicación con altísima exigencia en rendimiento en tiempo real, especialmente en:

- Scroll infinito con contenido multimedia
- Reproducción de video (Reels, Stories)
- Interacciones en vivo (likes, comentarios)
- Uso intensivo de cámara y procesamiento de imágenes

Desde una perspectiva de ingeniería:

- Control total del hardware:
  El desarrollo nativo permite acceso directo a GPU, cámara, sensores y optimización de memoria.
- Menor latencia:
  Evita capas intermedias como el "puente" de React Native o el motor intermedio de Flutter.
- Optimización específica por plataforma:
  Instagram puede adaptar el comportamiento según iOS o Android, maximizando rendimiento en cada uno.
- Escalabilidad del rendimiento:
  Al manejar millones de usuarios concurrentes, pequeñas mejoras a nivel nativo tienen gran impacto.

Instagram sí ha usado React Native en partes, pero no como base total.

Razones técnicas:

- Limitaciones del puente (React Native):

    - Comunicación JS ↔ nativo puede generar latencia.
    - No es ideal para animaciones complejas o video en tiempo real.
- Flutter (motor propio):
    - Introduce una capa adicional de renderizado.
    - Puede dificultar integración profunda con APIs nativas críticas.
- Costo de migración:
    - Reescribir una app tan grande implica riesgo operativo alto.

Los principales desafios que presenta instagram son:

Principales desafíos:

- Renderizado eficiente de listas enormes (feeds):
    - Uso de técnicas como *virtualización de listas*.
    - Carga diferida (lazy loading).
- Procesamiento de video en tiempo real:
    - Requiere uso intensivo de GPU.
    - Compresión, buffering y sincronización.
- Animaciones fluidas (60 FPS o más):
    - Transiciones entre historias, reels, etc.
    - Necesidad de minimizar bloqueos del hilo principal.
- Gestión de memoria:
    - Evitar leaks al cargar imágenes/videos constantemente.

Si Instagram fuera 100% React Native:

- Problema principal: el bridge (JS ↔ nativo)

Cuellos de botella:

- Scroll de miles de publicaciones:
    - Cada interacción pasa por el bridge → posible lag
- Animaciones complejas:
    - No siempre se ejecutan en el hilo nativo → pérdida de FPS

- Eventos de alta frecuencia:
    - Likes, comentarios en tiempo real → saturación del bridge

- **Waze**
    - **Ingeniería Inversa de la Decisión:** Waze apuesta por una arquitectura Nativa con componentes en C++ debido a la carga computacional que implica el renderizado de cartografía dinámica y el procesamiento de señales GPS cada segundo. Flutter o React Native añadirían una capa de interpretación que elevaría el consumo de batería y latencia de respuesta, factores críticos para un conductor.
    - **Análisis de Rendimiento y Arquitectura:** El desafío es renderizar miles de polígonos (edificios, calles, carros) a 60 FPS. Waze resuelve el cuello de botella procesando la telemetría del GPS de forma asíncrona. Al no depender de un "puente" (como React Native), la comunicación entre el motor gráfico y la ubicación del usuario es inmediata.
    - **Desafío de Nuevas Pantallas (AR):** Ante la llegada de lentes de realidad aumentada, la arquitectura de Waze tiene mayor facilidad de adaptación. Su motor ya está optimizado para cálculos espaciales de baja latencia. En cambio, Flutter o React Native dependerían de actualizaciones de terceros para soportar el renderizado 3D de bajísima latencia que exige la realidad aumentada interactiva.
    - **Veredicto Crítico:** Como ingenieros, para una app de mapas 3D y notificaciones de alta frecuencia en dispositivos plegables, elegiríamos **Nativo**.

      Fundamentación: La arquitectura nativa permite una gestión de memoria más granular y un control preciso sobre cómo se redibuja la interfaz al cambiar el tamaño de la pantalla (foldables), garantizando que el motor gráfico no pierda el estado de la navegación.

- **¿Qué es un aplicativo nativo, hay varios, como se clasifican?**

Es una aplicación desarrollada para una plataforma específica (iOS o Android) utilizando su SDK (Software Development Kit) oficial. Se caracteriza por compilarse directamente en código binario que el procesador del teléfono entiende sin necesidad de intérpretes, lo que permite aprovechar al máximo el hardware y los recursos del sistema.

Clasificación de Arquitecturas Nativas:

- **Nativo Puro:** Es el estándar, utiliza lenguajes de alto nivel como Kotlin/Java (Android) y Swift (iOS). Es la opción con mayor soporte y estabilidad a largo plazo.
- **Nativo de Bajo Nivel (NDK):** Aplicaciones que integran librerías en **C/C++** para procesos críticos. Es la arquitectura de **Waze**, necesaria para el procesamiento intensivo de datos geográficos y renderizado 3D de alta velocidad. Es nativo, pero de bajo nivel.
- **Nativo Multiplataforma (Compilado):** Frameworks como Flutter que, aunque permiten un solo código fuente, generan un binario nativo. Se diferencian de los anteriores en que no siempre utilizan los componentes de interfaz estándar del sistema operativo, sino motores de renderizado propios.